

ATTORNEY SHIELD

ASI 2.0 - Native Android & iOS

Attorney Shield 2.0 - Backend asks from

Backend asks — re-verified 2026-08-13

Colour system: Attorney-Shield Color Scheme Review

Repository: samson-phillip/asi-claude

DRAFT FOR APPROVAL

Contents

Where things stand after your latest pass	3
At a glance	3
Already working - please don't re-do these	4
Yes A1, A2 and A3 are fixed - verified 2026-08-13	6
~~A1 - countries returns empty while the same rows resolve by id~~ · FIXED	6
~~A2 - Incident types are not seeded~~ · FIXED	7
~~A3 - Document types and fields are not seeded~~ · FIXED	7
A4 - Still no real case for the test member · MEDIUM	7
A7 - addSubaccount accepts a seatPriceID that does not exist · DATA	8
A8 - the attorney behind a call cannot be read without a timeout · MEDIUM	8
A5 - a member cannot delete their own document · HIGH	9
A6 - what does the document-field type vocabulary mean? · MEDIUM	9
B1 - Phone verification · HIGH · blocks screens 08, 09	9
B2 - Pronouns · LOW · blocks one field of screen 10	10
B3 - Situation preferences · HIGH · blocks screens 13B, 13C, 27B	10
B4 - Emergency-contact notify-by · MEDIUM · blocks two controls on screen 16	10
B5 - Trial and guest · HIGH · blocks 13 screens	10
B6 - No member-readable transcript · LOW · one link on screen 32	11
B8 - Notification categories, a frequency dial, and what kind means · MEDIUM	11
B7 - A member cannot change their card in the app · MEDIUM · blocks 33D	12
D1 - POST /api/vonage/video/member-call takes no authentication	13
E1 - Deep-link contract for the web->app handoff · HIGH	13
E2 - Domain verification files · HIGH · web hosting	13
What we ship as each lands	13
Where we are	14

From: mobile (Android + iOS) **Date:** 2026-08-13 - **re-verified end to end after "done with my end"**

Environment: <https://gateway-dev.attorneyshield.io/query> · <https://comms-dev.attorneyshield.io>

Every item below is something **you can act on**. Our own to-dos, design questions and the things we are arranging ourselves are deliberately not here.

Everything was verified by introspecting the schema and calling the operations as the test member - not inferred from the web client. Where we were wrong earlier, it is corrected and marked.

Where things stand after your latest pass

We re-ran every item in this document against dev today, signed in as the test member. **countries is fixed - thank you, that was the one you said you were working on**. Nothing else in here has moved yet, so the list below is the remaining work rather than a new list.

Re-tested today	Result
<code>countries { iso2 name }</code>	United States Yes - was <code>[]</code>
Schema shape	Byte-identical to yesterday: 238 queries, 297 mutations, no type or input-field changes
A5 <code>deleteUserDocument</code> on the member's own file	still forbidden
A6 field type vocabulary and the dev test rows	unchanged
D1 POST <code>/api/vonage/video/member-call</code> unauthenticated	still accepted (reaches body validation with no token)
A4 <code>casesByUser</code>	not empty any more - but only because our own test calls created six cases . <code>partnerID</code> is null on all of them and the jurisdiction is still the hard-coded dev seed, so attorney pre-selection is as untestable as before

We are not blocked on any of it right now - this week's screens are all built on operations that already work. Please treat the below as a queue, not an emergency.

At a glance

#	Ask	Severity	Unblocks
A1	<code>countries</code> returns <code>[]</code>	~~HIGH~~	Yes FIXED - verified today
A2	Incident types + an English default language	~~BLOCKING~~	Yes FIXED - Home renders real tiles

#	Ask	Severity	Unblocks
A3	Document types + fields	~~HIGH~~	Yes FIXED - the Glovebox is built
A4	Still no <i>real</i> case for the test member	MEDIUM	Attorney pre-selection, real jurisdiction
A5	<code>deleteUserDocument</code> is forbidden for a member's own file	HIGH	Removing a document from the Glovebox
A6	What the document-field type vocabulary means	MEDIUM	Rendering the right control per field
A7	<code>addSubaccount</code> accepts a <code>seatPriceID</code> that does not exist	DATA	Correct billing on family seats
A8	<code>attorneyAssignments { attorney { ... } }</code> times out	MEDIUM	Attorney name on the Activity timeline
B1	No phone send/verify operations	HIGH	Registration screens 08, 09
B2	No pronouns field	LOW	The last field of screen 10
B3	No situation-preference operations	HIGH	Screens 13B, 13C + the saved-three row on Home
B4	No notify-by field on emergency contacts	MEDIUM	Two controls on screen 16
B5	No trial or guest operations	HIGH	13 screens (V1-V2, T1-T8, G1-G3)
B6	No member-readable transcript	LOW	"View transcript" on Test Call entries (screen 32)
B8	Notification categories, frequency and kind	MEDIUM	Most of screen 26
B7	No way to change a card in-app	MEDIUM	Screen 33D, and the Update row on 33A
C1-C8	Eight answers we are currently guessing	MEDIUM	Token refresh, PIN gate, guest design
D1	<code>member - call</code> takes no authentication	SECURITY	-
E1-E2	Deep-link contract + domain files (not the gateway)	HIGH	Silent web->app handoff

Already working - please don't re-do these

Area	Operations
Sign in, password	login
Sign in, one-time code	<code>requestLogin0tp / verifyLogin0tp</code>

Area	Operations
Date of birth, gender	<code>updateMyProfile</code>
Home + mailing address	<code>updateMyProfile, createUserAddress</code>
Security PIN	<code>setMemberPin, memberPinStatus</code>
Set a password	<code>setPassword, User.hasPassword</code>
Emergency contacts	<code>createEmergencyContact + CRUD</code>
States/provinces	<code>subdivisionsByCountry</code> - all 50 US states resolve
Video call credentials	<code>POST /api/vonage/video/member-call</code>
Membership + entitlement	<code>myMembership, membershipEntitlement</code>
Plan name and price	<code>Membership.items -> price -> product</code>
Card on file (read)	<code>myPaymentMethods</code>
Family sub-accounts	<code>mySubaccounts, addSubaccount, removeSubaccount, sendUserInvite</code>
Receipts	<code>invoicesBySubscriber</code>
Call history	<code>commsCallsByMember</code>
Change password	<code>changePassword</code>
Language + notification prefs	<code>updateMyProfile (primaryLanguageTag, notificationsEnabled, marketingOptIn)</code>
Terms and privacy text	<code>adminTermsOfServiceList</code> - four live documents
Notifications	<code>notificationList, unreadNotificationCount, markNotificationRead, markAllNotificationsRead, clearNotifications</code>
Close account	<code>deleteMyAccount</code>

All of the above are **built and writing to dev**. The Vonage SDK is also proven on a real Android device: the native library loads, reports 2.32.1, and completes a round trip to Vonage.

The second half of that table is new since the last version of this document. We had not looked past registration; when we did, most of the account section turned out to already exist. **We are telling you so you don't build it twice** - the member-facing Profile, Payment & plan, Family members, Settings and Activity screens are all built on operations that were already there.

A. Environment and seeding

Yes A1, A2 and A3 are fixed - verified 2026-08-13

Confirmed against dev as the test member, and the apps now render it:

Check	Result
countries	United States Yes
adminIncidentTypeList(activeOnly: true, countryISO2: "US")	6 types - Test Call, Traffic Stop, Domestic, Auto Accident, Pedestrian Stop, Other Yes
adminLanguageList	en-US with isDefault: true, plus es-ES and ar-SA Yes
Translations	English and Spanish on every type Yes
iconFilePath	Real CloudFront URLs on all six Yes
adminDocumentTypeList	Driver's Information, Health Information, Gun Information, Citizenship Info - all four Glovebox sections Yes
adminDocumentFieldList	20 fields Yes

Home now shows six real incident tiles with their English names for the first time. That was the blocking item; it is gone.

Two small notes, neither urgent:

- 1 The code values are human strings** - "Traffic Stop", not `traffic_stop`. We had assumed `snake_case`, so our icon lookup matched nothing and every tile fell back to a generic shield until we normalised it. Only flagging in case another client made the same assumption.
- 2 adminDocumentTypeList also contains what look like test rows** - "contract 2" and "Extra books", alongside the generic Contract / Form / Guide / Policy / Incident Report. We will render only the four Glovebox sections, but you may want to tidy those out of dev.

The rest of this section - A4 - is still open.

~~A1 - countries returns empty while the same rows resolve by id~~ · FIXED

Correction to what we told you earlier. We previously said no countries were configured. That was wrong:

Query	Result
countries { iso2 }	[]
country(id: "f989d06a-8813-11f1-a446-06cf81ac74a7")	United States, US Yes
subdivisionsByCountry(<that id>)	all 50 states + DC Yes

The data exists. The **list query** is what comes back empty, and our test member's own profile points at that same US id.

We work around it by reading the country from the member's profile instead of offering a picker - so it is not blocking us - but the list query looks broken or scoped in a way nobody intended, and we would rather tell you than quietly route around it.

~~A2 - Incident types are not seeded~~ · FIXED

```
adminIncidentTypeList(activeOnly: false) -> []
adminIncidentTypeList(activeOnly: true, countryISO2: "US") -> []
adminLanguageList -> 1 entry: ar-SA, isDefault: false
```

Retested **while authenticated**, with and without a country filter, so this is not a side-effect of A1.

This is the one that hurts most. The home screen shows "No incident types are configured yet.", so we cannot exercise the incident tiles, the attorney chip row, or place a call with a real incident type. Finishing onboarding currently lands the member on an empty screen.

Need: incident types seeded with translations - the design uses Traffic Stop, Auto Accident, Pedestrian Stop, Domestic, Test Call, Other - plus **an English language entry marked isDefault: true**. Without the language, our label resolution (English -> org default -> first available -> humanized code) falls through to Arabic.

~~A3 - Document types and fields are not seeded~~ · FIXED

```
adminDocumentTypeList -> []
```

The operations all exist - adminDocumentFieldList, requestUserDocumentUpload, createUserDocument, userDocumentList, userDocumentDownloadUrl, and saveUserDocumentValue for plain text fields. So this is a **seeding** ask, not a build ask.

Every upload and every saved value is keyed by adminDocumentFieldsId. With no types there are no fields, so there is nothing to attach a document or a value to, and none of the Glovebox can be built.

Need: the four sections seeded with their fields, per the design:

Section	Fields the design shows
Driver's Information	licence document upload
Health Information	three plain text fields + document upload
Gun Information	two yes/no questions, permit number, issue state
Citizenship Info	plain fields + document upload

Also please tell us **what fieldType values exist**, so we render the right control for each field rather than guessing.

A4 - Still no *real* case for the test member · MEDIUM

casesByUser is no longer empty, but nothing was seeded - **the six cases there are ones our own test calls created**. All six look like this:

```
jurisdictionID: de400000-0000-4000-8000-000000000001 <- the hard-coded dev seed
partnerID: null
```

partnerAttorneys returns [] for both the dev partner id and the org id, so attorney pre-selection still cannot be exercised at all - which was the point of the ask.

Need: one case on the test member with a real jurisdiction and a real partner, and at least one attorney under that partner.

If it is easier to point us at an environment that already has this, that works just as well. We only need somewhere to develop against.

A7 - addSubaccount accepts a seatPriceID that does not exist · DATA

Found by accident while probing permissions. We sent an all-zeros UUID expecting a validation error:

```
addSubaccount(input: {
  organizationID: "<org>", firstName: "Probe", lastName: "Only",
  email: "probe-only@example.invalid",
  seatPriceID: "00000000-0000-0000-0000-000000000000" # does not exist
})
-> { "id": "33d1fdb4-..." } # created anyway
```

The sub-account was created with kind: "included", status: "invited". We removed it immediately with removeSubaccount and confirmed mySubaccounts is empty and the seat counts are back to seatsUsed: 1.

Blank names or a blank email are rejected (billing: first name, last name and email are required), so validation exists - the price id simply is not checked. A client that passes the wrong id gets a seat that bills against nothing.

Need: reject an unknown seatPriceID. (We pass the real one from myMembership.items[].price.id, so this is not blocking us.)

A8 - the attorney behind a call cannot be read without a timeout · MEDIUM

For the Activity timeline we want the attorney's name on each session, as the design shows. The nested field hangs:

Selection	Result
attorneyAssignments { id status }	1.7 s Yes
attorneyAssignments { id status attorneyId }	1.1 s Yes
attorneyAssignments { id attorney { displayName } }	504 after 60 s

attorneyId resolves (de300000-0000-4000-8000-0000000020006) but there is no member-callable query that turns an attorney id into a name - partnerAttorneys is empty for us (A4).

We ship the timeline without attorney names rather than showing an id or leaving a blank where a name belongs.

Need: either the nested resolver fixed, or a name on the assignment itself.

A5 - a member cannot delete their own document · HIGH

The Glovebox is built and uploading against dev. One thing does not work:

```
mutation { deleteUserDocument(id: "<the member's own document>") }
-> {"errors":[{"message":"forbidden"}]}
```

Called as the signed-in member, on a document that member uploaded a moment earlier. Everything else in the chain works - requestUserDocumentUpload -> PUT to S3 -> createUserDocument -> userDocumentList -> userDocumentDownloadUrl all round-trip, and we fetched the exact bytes back.

The design puts a `` on every uploaded document tile (screens 14A, 14C: *"Deleting the last file restores the dropzone"*). **We have shipped no delete control**, because one that always fails is worse than none.

What we need: either member scope on deleteUserDocument for their own rows, or a deleteMyUserDocument-style operation.

Meanwhile: two test files of ours are stuck on the test member's Driver's Information section (probe.txt, ui.xml) and we cannot remove them.

A6 - what does the document-field type vocabulary mean? · MEDIUM

AdminDocumentField.type is a free-form string. Across dev we see:

```
text dropdown file image agreement policy guide form
example "example 3" "example 4" "example 5"
```

We render text, dropdown and file, and treat everything else as not-member- input. That rule is what keeps the organisation's templates (Contract, Form, Guide, Policy, Incident Report) out of a member's personal Glovebox.

image is the one that bit us. We first mapped it to a file upload - it looks like one - and "Policy" duly appeared in the member's Glovebox, because Policy has an image field called "Example". We now treat image as unknown.

What we need: the intended list of type values and what each should render as. If image really is an upload, we will map it - we just will not guess.

Also, adminDocumentTypeList still holds dev test rows (contract 2, Extra books) and several example* fields. Harmless to us now, but worth tidying.

B. Operations that do not exist

We searched all **238 queries and 297 mutations** for each of these. Where a backend piece is missing, we have **left the screen unbuilt and listed it here** rather than shipping a placeholder that looks finished.

B1 - Phone verification · HIGH · blocks screens 08, 09

- updateMyContactInfo(input: { phoneE164 }) **stores** a number. That works.
- **Nothing sends or checks a phone code.** No requestPhoneOtp, no verifyPhone, nothing equivalent.

- `requestLoginOtp(channel: SMS)` is a **sign-in** code sent to an *already-verified* phone, so it cannot verify a new number - it is circular.
- `phoneVerifiedAt` is writable only via `CreateUserInput` / `UpdateUserInput`, which are admin operations, not member self-service.

Need: two operations a signed-in member can call - send a code to a supplied number, and verify that code, setting `phoneVerifiedAt` on success.

Also confirm the code length. Sign-in codes are 4 digits; the design specifies 6 for phone verification.

We deliberately did not ship screen 08 alone. Its own sub-line promises "We'll text a code to verify it", and rewording that to hide the missing half would leave a feature that looks finished and never gets revisited.

B2 - Pronouns · LOW · blocks one field of screen 10

Date of birth and gender are both on `UpdateMyProfileInput`. Pronouns are not, and no input type in the schema has a field matching pronoun.

Need: one nullable string on `UpdateMyProfileInput` (and on `UserProfile` if you want it readable).

B3 - Situation preferences · HIGH · blocks screens 13B, 13C, 27B

The member's saved "three most common situations". Nothing matching situation, preference or favourite anywhere.

Need: a read and a write, capped at three incident-type IDs per member. Roughly:

```
myCommonSituations: [ID!]!
setMyCommonSituations(incidentTypeIds: [ID!]!): Boolean
```

Home currently shows the full incident list because there is nowhere to store a choice of three.

B4 - Emergency-contact notify-by · MEDIUM · blocks two controls on screen 16

The design collects *how* each contact is alerted: "Notify them by - Text message / Email / Choose one or both".

`CreateEmergencyContactInput` has `userId`, `name`, `relationship`, `phoneE164`, `email`, `isPrimary`, `notes`. There is **no notify-by field**, and nothing matching `notify` on any input type.

We could have written it into `notes`, but that is free text being used as a settings column - it works until something reads `notes` expecting `notes`.

Need: two booleans (`notifyBySms`, `notifyByEmail`) or a small enum on the contact.

B5 - Trial and guest · HIGH · blocks 13 screens

Nothing matching `trial` or `guest`.

The design has a 7-day limited trial with an in-app conversion gate (V1-V2, T1-T8) and a guest mode entered from an unrecognised email at sign-in (G1-G3).

The question that decides the whole design: is a guest a real account with a role, or purely local state?

Note also that `requestLoginOtp` **deliberately does not enumerate accounts** - an address with no account still returns `sent: true` with the submitted address masked back. That is good security, and it means **the app cannot detect an unrecognised email at sign-in**, so guest mode cannot be entered the way the design describes. Whatever the model turns out to be, that branch needs rethinking with you.

B6 - No member-readable transcript · LOW · one link on screen 32

The Activity design deliberately drops recording replay, but keeps *"Test Call entries keep View transcript, the intended way to review demo sessions."*

`CommsCallRecording` exists with `playbackUrl` and `vonageRecordingUrl`, but those are recordings, not transcripts, and `biCallRecordingUrl` is a BI operation a member cannot call. Nothing matching transcript exists.

We ship the timeline without the link. Not urgent - but if transcripts are coming, we would rather wire the real thing than add a link now.

B8 - Notification categories, a frequency dial, and what `kind` means · MEDIUM

The notification chain itself works, and is correctly scoped - we verified `create`, `list`, `unreadOnly`, `mark one`, `mark all` and `clear`, and confirmed that reading or writing another member's inbox returns `forbidden`. Screens 22, 23, 24 and 25 are built on it. Thank you; nothing here is blocking.

Screen 26 is the exception. The design has **four category toggles and a frequency dial**:

Design control	Field we could store it in
Setup reminders	-
Tips & know-your-rights	-
Account & billing	-
Safety-critical (always on)	n/a - nothing to store
How often: Occasionally / Rarely / Off	-

`UserProfile` has exactly two booleans: `notificationsEnabled` and `marketingOptIn`. **We did not map "Tips & know-your-rights" onto `marketingOptIn`**. That flag is a marketing-consent record with legal weight, and quietly relabelling a consent as a content preference is the kind of thing that is discovered during an audit. It is shown as "Marketing emails", which is what it is.

Need: either per-category booleans and a frequency enum on the profile, or a decision that one master switch is the product.

Also: what is `kind` supposed to be? `createNotification` accepted a kind of "totally-made-up-kind" without complaint. We carry it through and group loosely rather than switching on it - the document-field type already taught us what guessing costs (A6) - but we would rather render the right icon and grouping than a generic one.

One more, small: `createNotification` is callable by a member for their own inbox. Correctly scoped (other users are refused), so it is not a hole - just odd, and worth a look in case it was meant to be admin-only.

B7 - A member cannot change their card in the app · MEDIUM · blocks 33D

Screen 33D lets a member edit the expiry and billing ZIP inline and replace the card. Today:

- **No** `updatePaymentMethod`. The mutations are `createPaymentMethod`, `attachPaymentMethod`, `setDefaultPaymentMethod`, `detachPaymentMethod`.
- **No billing ZIP** on `PaymentMethod` at all - the fields are `brand`, `last4`, `expMonth`, `expYear`, `billingName`, `billingEmail`, `billingCountryID`.
- `attachPaymentMethod` **takes a** `providerRef` - a token from the payment provider. Minting one needs the provider's SDK and publishable key in the app, which is a decision for you, not something we should choose unilaterally.

We show the card read-only on Payment & plan - brand, last four, expiry - with no Update control, because every write path available to us either does not exist or would need a credential we have not been given.

Need, in order of preference: (a) a hosted card-update link we can open, the same way checkout already works on the web; or (b) `updatePaymentMethod` for expiry, plus a ZIP field; or (c) the provider's publishable key and a decision to embed their SDK.

C. Answers we need - each of these is a guess today

- 1 **countryISO2 on** `verifyLoginOtp` - we send null, because it is optional and that is what the web client sends. But what is it *for*? If it affects routing or jurisdiction we would rather send something real.
- 2 **refreshToken - access-token lifetime, and does refreshing rotate the refresh token?** This matters more for us than for web: a browser tab is short-lived, but a native app sits backgrounded for days, and this is an app people open during a police encounter. **We have not wired refresh yet because of this.**
- 3 **Is** `verifyMemberPin` **the intended server-side gate for ending a call?** The design says the PIN's only job is ending a live session securely - it does not unlock the app or protect recordings.
- 4 **Who sends the emergency-contact alert?** The design says contacts are "alerted with your location the moment you connect to an attorney". We only ever call `member-call`, so it must be server-side - but if the app is meant to trigger it, we need an operation.
- 5 **Is one-device-at-a-time deliberate and permanent?** `LoginPayload` returns `otherSessionsRevoked` and `mySessionStatus` reports `another_device`, so signing in on the web ends the app's session and vice versa. We can live with it; it is worth confirming for a phone people open during an encounter, since signing in on a laptop signs out the phone in their pocket.
- 6 **Casing is inconsistent and we follow whatever each operation asks for** - the gateway mixes `userID` (`login`, `casesByUser`) with `userId` (`setMemberPin`, `updateMyProfile`'s `countryId`), and comms REST uses `memberUserId`. Worth settling before anyone adds a field.
- 7 **Sign-in codes are 4 digits** - confirmed live, and we have built for 4. Flagging only because the design specifies six-cell entry; we assume that describes phone verification (B1), which is a different code.

- 8 **Is there, or could there be, an "onboarding complete" flag?** We currently infer completeness from date of birth + address + PIN + password + contacts. A real flag would be more honest than our inference.
-

D. Security

D1 - POST /api/vonage/video/member-call **takes no authentication**

Anyone can place a call against any organizationId / memberUserId they can guess, and that call routes to a **real attorney**.

Nothing we have built depends on it staying open, and we are not designing around it. Flagging rather than assuming it is known.

E. Not the gateway - please forward these

Both are blocking us, but neither is a backend/GraphQL task. Included so they do not fall between teams.

E1 - Deep-link contract for the web->app handoff · HIGH

Screens 07 and T4 hand the member back to the app "with email pre-filled", but nothing records the actual path or parameter names.

Currently implemented: we accept /app/return, /return-to-app and /app on attorney-shield.com and www.attorney-shield.com, reading an email query parameter. It is confined to one file, so a confirmed contract is a one-line change.

Please do not design the link to carry a credential. We treat it as untrusted input - anyone can send a link. The email is a text-field prefill only, and we have a test asserting that a link carrying accessToken, userID or roles yields nothing but the email.

E2 - Domain verification files · HIGH · web hosting

For a link to open the app *silently*, both hosts need:

- **Android:** /.well-known/assetlinks.json with our signing-certificate SHA-256 fingerprint for com.app.attorney.shield
- **iOS:** /.well-known/apple-app-site-association for our Team ID + com.app.attorney.shield

Until these exist, Android shows a "which app?" chooser and iOS universal links do not fire at all. **We owe you the fingerprint and Team ID** once our release keystore exists - that part is on us.

What we ship as each lands

You give us	We ship
~~A2 - incident types + an English language~~	Yes done - Home renders real tiles
~~A3 - document types + fields~~	Yes done - the Glovebox is built and uploading
A5 - member scope on delete	The `` on each document tile
A6 - the type vocabulary	Confidence that every field renders as intended
A4 - a case for the test member	Real jurisdiction and attorney pre-selection
A8 - the attorney behind a call	Attorney names on the Activity timeline
B6 - a transcript	"View transcript" on Test Call entries
B8 - categories + frequency	The rest of screen 26's controls
B7 - a card-update path	Screen 33D and the Update row on Payment & plan
B1 - phone send/verify	Registration screens 08 and 09
B2 - a pronouns field	The last field of screen 10
B3 - situation preferences	Screens 13B, 13C and the saved-three row on Home
B4 - notify-by	The last two controls of screen 16
B5 - a decision on the guest model	Trial and guest scoped, 13 screens
C2 - token lifetime and rotation	Token refresh, so sessions survive being backgrounded
C1, C3-C8 - the remaining answers	Guesses removed from code we have already shipped
E1 - the deep-link contract	A one-line change, then verified end to end

Where we are

Built and verified against dev: the welcome carousel, sign-in by password **and one-time code**, the profile checklist and its readiness card on Home, personal details, address, security PIN, set-a-password, emergency contacts, **a home screen with six real incident tiles**, the call flow up to "connecting", and the deep-link handler.

The Glovebox is built and verified on a device against dev - sections and fields render from your configuration rather than being hardcoded, text and dropdown answers save, and a file goes picker -> presigned S3 -> `createUserDocument` with the tile appearing straight afterwards.

One thing we have never been able to prove: a video call actually going live. `member-call` returns 409 no attorney is available on dev. We are arranging an attorney ourselves by running the desktop app - not an ask - but until that happens, the step from "connecting" to a live picture is the one part of the product still taken on faith.